INTRO TO SPEECH PROCESSING · COURSE PROJECT

Bidirectional Speech-to-Speech Agent for Low-Resource Indic Languages

Fine-tuning Whisper-Turbo and integrating Indic Parler-TTS for Marathi, Gujarati, and English

मराठी Marathi | ગુજરાતી Gujarati | English

Ramnarayan Choudhary

MBZUAI · April 2026

Why I built this

GOAL

Build the full speech-to-speech loop end-to-end

A hands-on exercise that touches every part of the speech-processing stack — voice activity detection, language identification, ASR, fine-tuning, post-processing, and synthesis.

LANGUAGE CHOICE

Marathi and Gujarati

Both are widely spoken but weakly served by current open speech systems. Native context for me, and a useful low-resource testbed for adapter-based fine-tuning.

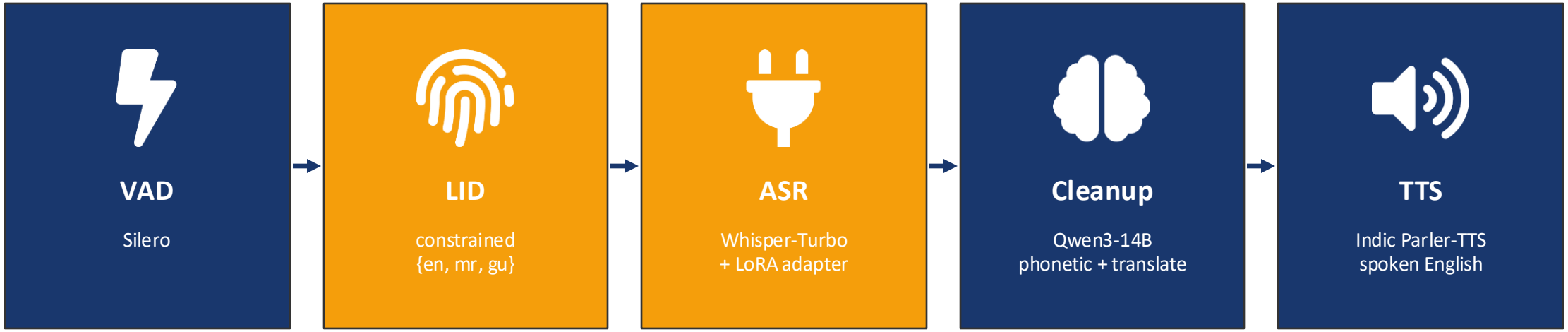
SYSTEM SHAPE

Bidirectional — audio in, audio out

A unidirectional ASR-only system would have left half the stack untouched. Going bidirectional forced engagement with TTS as well.

System at a glance

Five stages from raw audio to spoken English



Audio enters at 16 kHz. The first three stages do the speech understanding — VAD removes silence, LID picks the language, the ASR backbone with the matching LoRA adapter produces text. The last two stages do the language and speech generation — the LLM cleans the transcript and translates, and the TTS engine speaks the result.

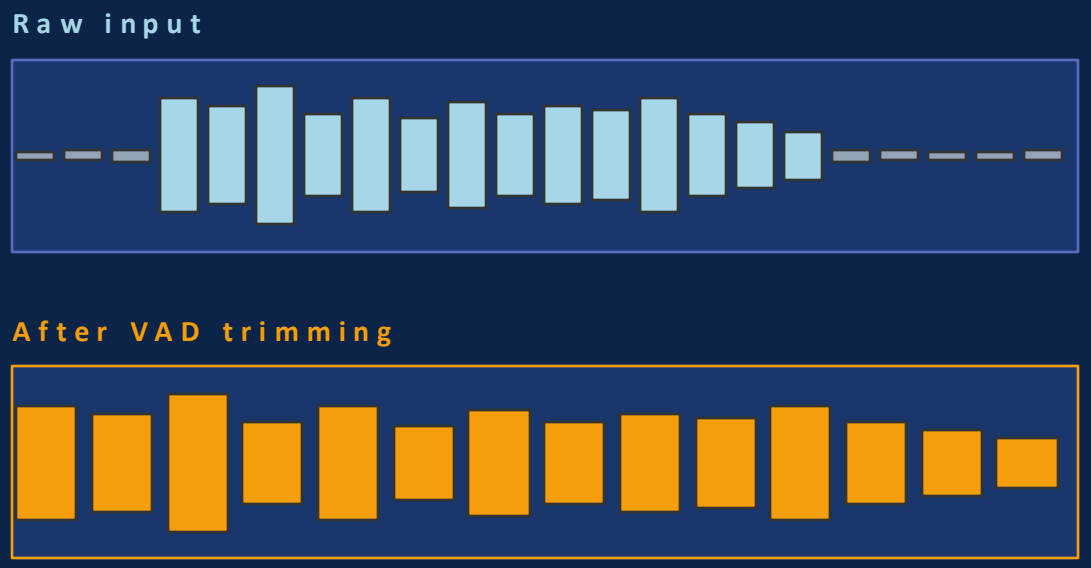
Where the pre-filter sits — and why

Voice activity detection ahead of the encoder

Whisper hallucinates on silence and on low-energy noise frames. It will produce plausible but invented text when fed extended quiet regions.

A lightweight VAD upstream of the encoder removes a class of errors that the acoustic model cannot recover from on its own. It is one of the few interventions in the pipeline that strictly subtracts errors rather than trading them for others.

Observed on the FLEURS clips: 1–3 seconds of non-speech trimmed per recording.



The frames that reach the encoder are the ones that carry information. Everything that would have triggered phantom decoding is already gone.

Narrowing the search space

Language identification under a known operational set

The default Whisper LID searches across 99+ languages. Many of those — Hindi, Sanskrit, Nepali — are phonotactic neighbors of Marathi and Gujarati and routinely win the argmax on real audio.

If the operational set is known, that openness is a liability. Restricting the candidate set to {English, Marathi, Gujarati} turns LID into a 3-way decision under exactly the same decoder, with no extra parameters.

The base model is consulted with the LoRA adapters disabled, so the routing signal is not biased by an already-active language adapter.

EMPIRICAL SIGNAL

> 20

logit-point gap on English vs everything else

Routing for English is effectively deterministic.

0.04

logit-point gap between Marathi and Gujarati on a real clip

That is the acoustic ceiling — the two languages are too close phonotactically for a decoder-logit comparison alone.

Per-language adapters via PEFT-LoRA

Backbone frozen, low-rank adapters trained per language

> *Full fine-tuning of an 809M backbone on small low-resource data risks catastrophic forgetting and is computationally heavy. PEFT-LoRA attaches small low-rank matrices to the attention projections — the backbone stays frozen, each language gets its own adapter, and adapters are switched at runtime.*

FROZEN BACKBONE

Whisper-Large-v3-Turbo

encoder retained · decoder distilled to 4 layers

fp16 precision · PyTorch SDPA attention

LoRA(en)
base only

LoRA(mr)
Marathi adapter

LoRA(gu)
Gujarati adapter

Adapters resident in GPU memory together

Switching between them is a runtime flag, not a weight copy. Sub-millisecond transition.

Adapter configuration

Parameter	Value
Rank (r)	32
Alpha (α)	64
Dropout	0.05
Target modules	q_proj, k_proj, v_proj, o_proj

Training

Item	Setting
Dataset	IndicSpeech 2022 · per-language splits
Optimizer	AdamW · lr = 1e-3 · linear decay
Hardware	NVIDIA RTX 5000 Ada · 32 GB · fp16
Steps	up to 1000 · checkpoint every 100

An LLM as a phonetic-error correction layer

Recovering from systematic acoustic mistakes using language priors

Raw STT output makes systematic mistakes — confused aspirated and unaspirated stops, wrong anusvara placement, character substitutions that change meaning.

These are precisely the kinds of errors a large language model with strong Indic-orthography priors knows how to fix. The LLM is asked to do two things at once: clean the native-script transcript and translate it to fluent English.

```
[CLEANED]: corrected native-script text
[ENGLISH]: fluent English translation
```

OBSERVED CORRECTIONS

Errors recovered by the LLM that the acoustic model produced

Acoustic output	Corrected	Meaning
कीवा	किंवा	“or”
विच्यार	विचार	“thought”
उप्योग	ઉપયોગ	“usage”

Optional: a domain hint can be supplied to disambiguate homophones — e.g. "news on agriculture" — and is concatenated to the system prompt.

Bringing Indic Parler-TTS into the loop

Two encoders, two tokenizers, one synthesis path

> *Indic Parler-TTS expects the spoken text and the voice description through different tokenizers. Routing them naively through the same path collapses generation to a default voice. The integration work was making the two paths coexist correctly.*

The problem

Two latent encoders with incompatible tokenization

- Transcript encoder expects native Indic script through an Indic tokenizer
- Style encoder expects English voice description through a flan-T5-large tokenizer

Sending the style description through the Indic tokenizer collapses generation to a single default voice with garbled audio.

The fix

Parallel tokenization paths

- Style description routed through a directly-instantiated flan-T5-large embedding layer
- Transcript routed through Parler's native Indic tokenizer
- Audio buffer upcast from fp16 to fp32 to avoid half-precision crashes during synthesis

Experimental setup

Datasets, hardware, and the metrics I tracked

Training data

IndicSpeech 2022

Per-language splits with separate training and validation. 16 kHz mono audio resampled and converted to 80-bin log-Mel features.

Evaluation benchmark

Google FLEURS — test split

Diverse speakers and news content. Out-of-distribution by design relative to the IndicSpeech training data.

Hardware

NVIDIA RTX 5000 Ada · 32 GB VRAM · fp16 · PyTorch SDPA

What I measured

STT accuracy

Word and Character Error Rates

Routing accuracy

did the LID select the right language

Translation BLEU

LLM output against FLEURS reference

TTS health

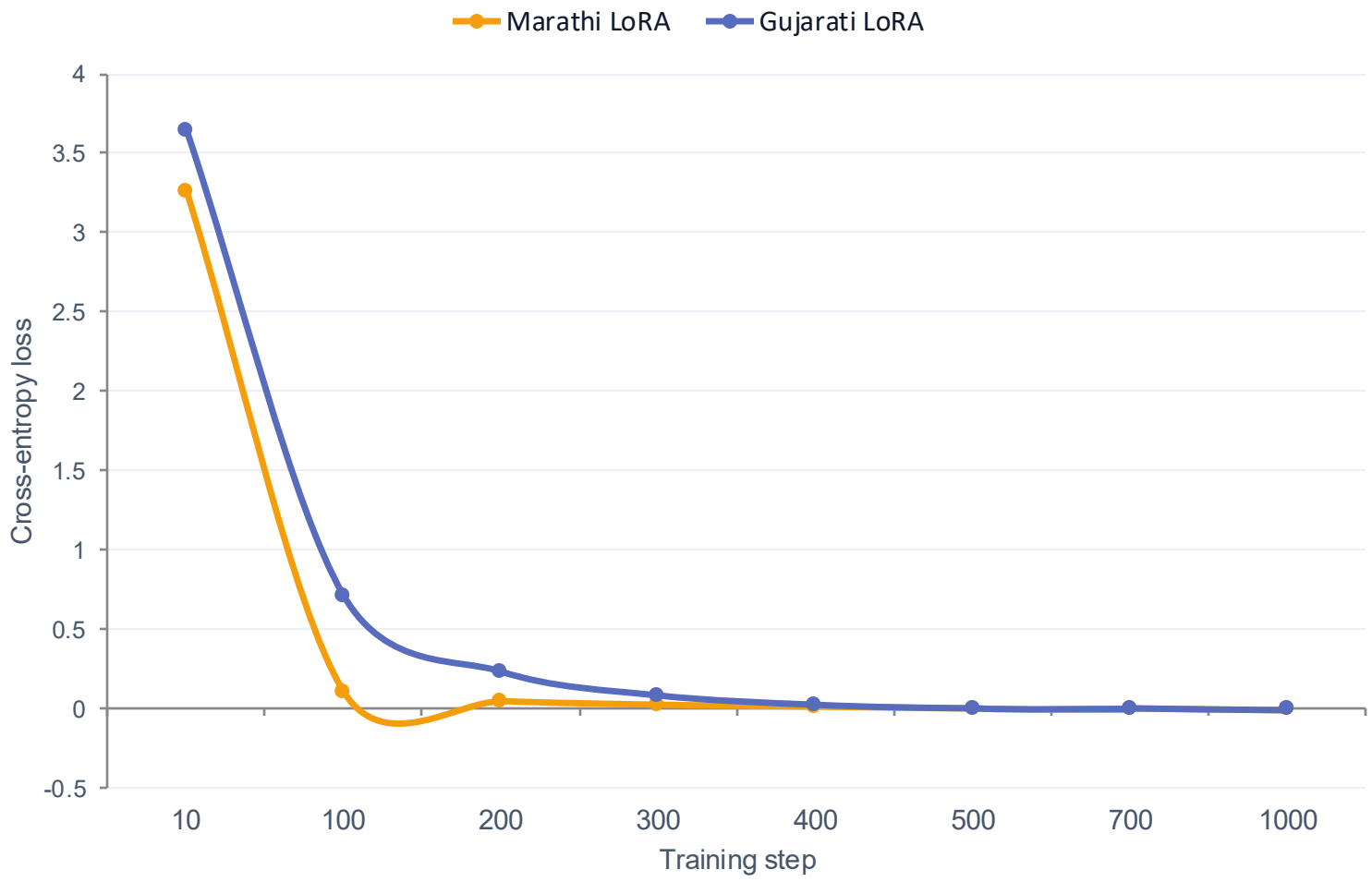
audio validity and generation latency

End-to-end latency

wall-clock per pipeline stage

Training convergence

Cross-entropy loss across 1000 LoRA fine-tuning steps



Final values

3.26 → 0.0018

Marathi loss at step 500

3.65 → 0.0016

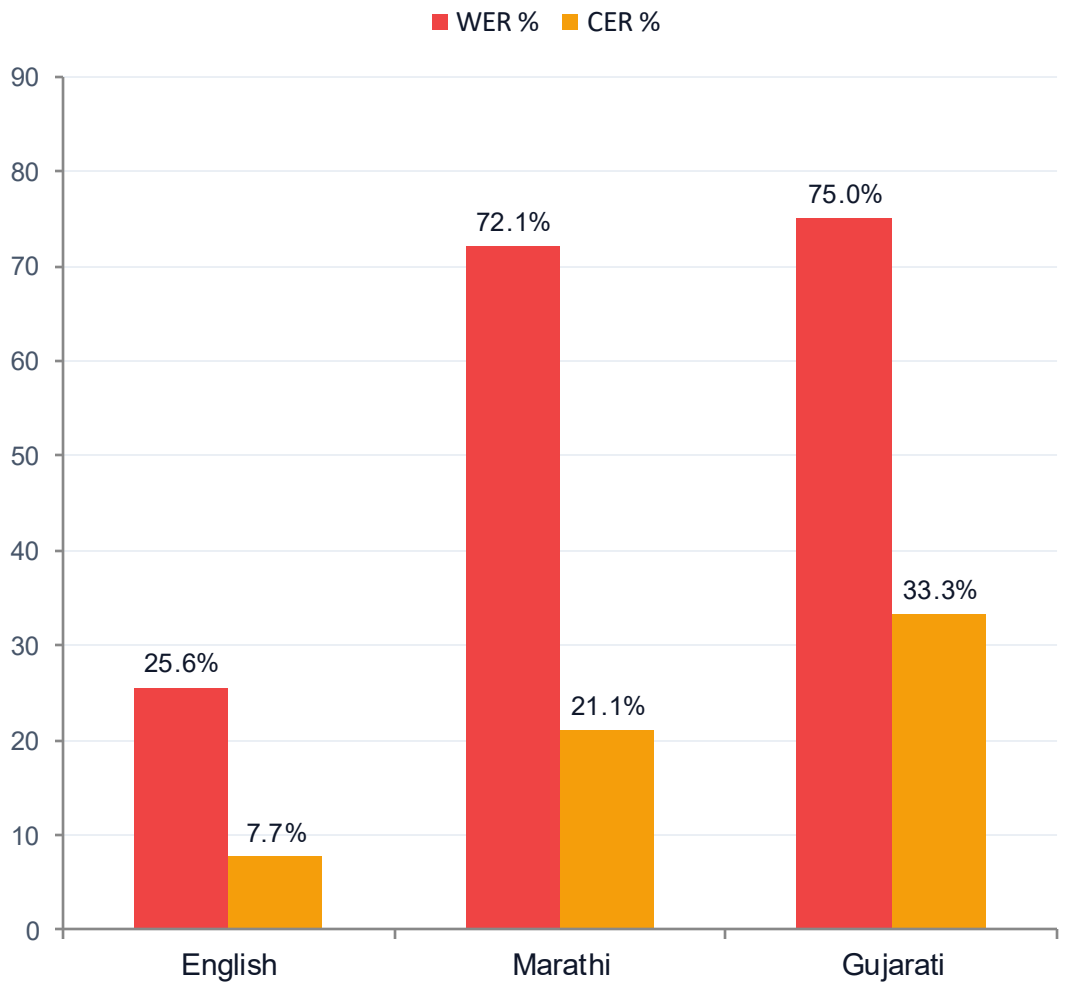
Gujarati loss at step 500

Both adapters converge cleanly. The genuine question is what survives a domain shift — addressed in the ablations.

STT accuracy on FLEURS

Word and Character Error Rates · n = 50 per language

Language	WER ↓	CER ↓	Latency	Model
English	25.6%	7.7%	1.4 s	Turbo (base)
Marathi	72.1%	21.1%	3.8 s	Turbo + LoRA(mr)
Gujarati	75.0%	33.3%	6.9 s	Turbo + LoRA(gu)



Reading the Indic numbers

FLEURS is out-of-distribution. On the IndicSpeech eval split the same adapters reach roughly 29% WER and 13% CER. The OOD gap is quantified explicitly in the next slide.

CER on Indic scripts

Marathi CER 21% means roughly 79% of characters are correct. Inflectional morphology — विच्यार vs विचार — inflates WER without proportionally damaging meaning.

Routing, translation, and latency

Three secondary metrics on the same FLEURS run

Autonomous routing · n = 30

Language	Accuracy	Correct / Total	Behavior
English	100%	10 / 10	20+ pt logit gap
Gujarati	60%	6 / 10	confused with Marathi
Marathi	50%	5 / 10	confused with Gujarati
Overall	**70%**	21 / 30	—

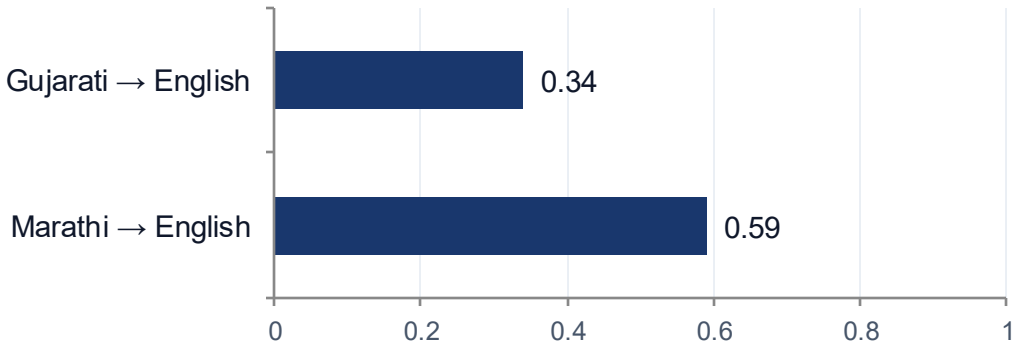
THE 0.04 LOGIT GAP

gu = 9.23 · mr = 9.16 · en = 5.98

Acoustic-only LID has a hard ceiling between Marathi and Gujarati. A dedicated acoustic classifier would be needed to break it.

LLM translation BLEU

Direction	BLEU	n
Marathi → English	0.59	10
Gujarati → English	0.34	10



END-TO-END LATENCY

mean 2.96 s · median 2.99 s · min 2.07 s · max 3.96 s · σ = 0.62 s → ≈ 3× real-time on a single GPU

Ablation A — what the LoRA actually fixed

Hallucination behavior and the in-domain vs out-of-distribution gap

Zero-shot Turbo vs LoRA-tuned Turbo · Gujarati agriculture-news clip

Zero-shot Turbo

Hallucinated content not present in the audio:

"...EPMC ... 500 people swept away by floodwaters ..."

Failure modes: invented sentence · EPMC mis-recognized · ghost text triggered by background noise

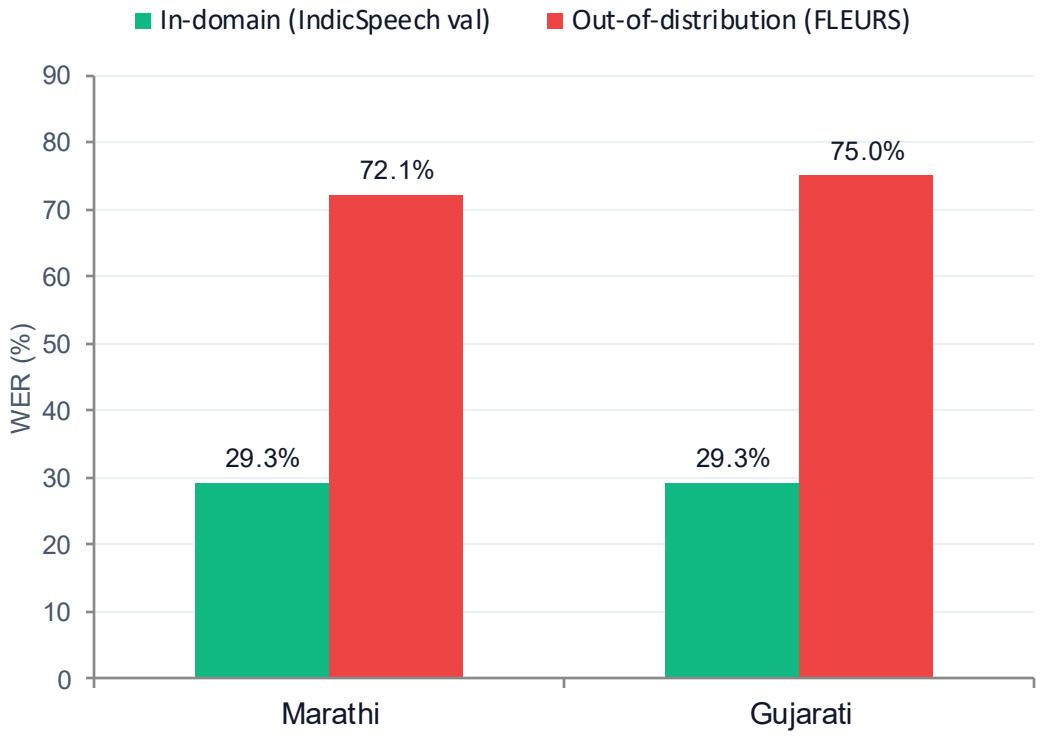
Fine-tuned Turbo + LoRA(gu)

Concise, grammatical, no ghost text:

"માર્કેટપીએમસીમાં પડેલા વરસાદને કારણે લાખો રૂપિયાનું નુકસાન થયું છે..."

Improvements: no hallucination padding · recognizes regional term "APMC Market"

In-domain (IndicSpeech val) vs out-of-distribution (FLEURS)

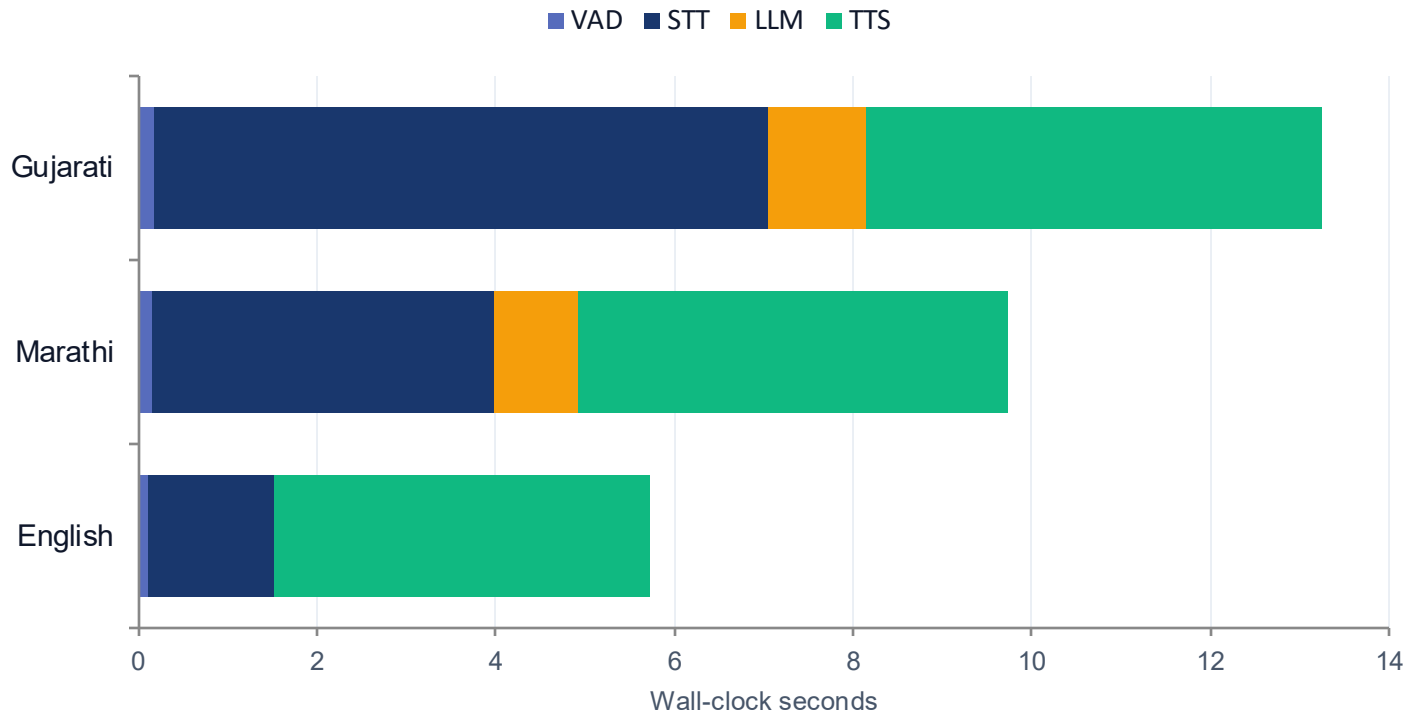


Δ ≈ 43–46 points · domain mismatch dominates the absolute error, not model capacity

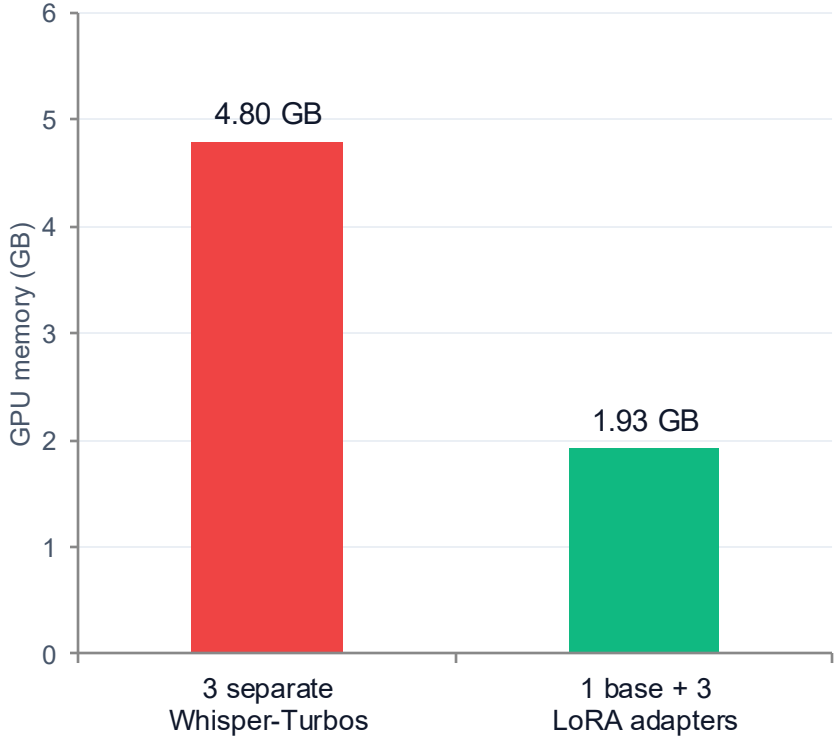
Ablation B — efficiency profile

Where time is spent and what the architecture costs in memory

Per-stage latency · mean over n = 50 FLEURS clips



Memory cost · one model per language vs shared backbone with adapters



Autoregressive TTS dominates · streaming TTS would be the largest single end-to-end latency win

60% reduction · a consequence of the adapter design, not the headline contribution

What this work shows

Four conclusions, in plain terms

01

The full speech-to-speech loop runs end-to-end for Marathi, Gujarati, and English.

Voice in, voice out. VAD, language identification, fine-tuned ASR, LLM cleanup, and Indic TTS all coexist behind a single application surface.

02

Per-language LoRA adapters on Whisper-Turbo are a workable shape for low-resource ASR.

Frozen backbone, small adapters trained per language, switched at runtime. They converge cleanly on small data and behave predictably under domain shift.

03

Marathi–Gujarati confusion is acoustic, not algorithmic.

Real audio produces decoder-logit gaps as small as 0.04 between the two. The right intervention is a dedicated acoustic LID head, not better prompting or constraints.

04

Autoregressive TTS is the dominant latency cost.

It is the largest single contribution to end-to-end wall time and the most natural place for a streaming or distillation-based intervention.

WHAT'S NEXT

Three concrete directions

Larger and more diverse training data

The OOD gap on FLEURS is the single largest source of absolute error. Common Voice and additional Indic corpora are the natural next step before any architectural change.

An acoustic LID head for Marathi vs Gujarati

Decoder logits cannot separate them — a small classifier over acoustic embeddings (for example ECAPA-TDNN) is the right tool.

A streaming TTS path

Replacing the current autoregressive synthesis with a streaming decoder removes the dominant latency stage and brings the loop closer to interactive use.